

Part 3 - Problem-solving

1. Your classifier turns out to be wrong about 30% of the time. What do you do — and how would you even decide whether it's “good enough” to be useful at all?

Answer:

First thing while building a classifier is to understand the cost of wrong prediction. Here 30% is wrong, so first check those false prediction and see if it comes from a particular class or any changes in input data(data dim change, or data from diff sensors etc.) or most of the false images comes from a new category that not exist in training data. If we observe this kind of pattern, best solution is retrain model with new data or more data representing wrong class.

Before retraining, we can change our review threshold and assign more prediction to review, so wrong prediction don't go out as final answer directly.

That is with new data distribution, we can also check the preprocessing layer if any pre processing done from our side affect model performance, like we use unnecessary data augmentation. Make sure that model output correctly assign to its corresponding label. Also inspect post processing layers to understand if anything wrong there that affect performance.

To understand what is wrong we need to inspect all components, not just model. In a production system any layer can introduce error.

For deciding good enough, find out does our confidence/margin or validation layer catch the wrong ones. So we need to check: split result into flagged(need_review) and not flagged, then check accuracy separately for both group. If not flagged group have high accuracy(like 90%+) and most wrong prediction fall in flagged group, then system is good enough to use, analyst can trust unflagged one and manually check flagged result. But if wrong prediction spread evenly in both group, then our flag not working properly, and our system is not trustworthy until that layer is fixed.

Also this depend on cost of wrong prediction in our use case - if it just help analyst do first pass, low risk. If it drive some automated decision without human check, then high risk.

2. This service runs offline, with no one watching it live. A month after deployment, how would you know it's still working correctly?

Answer:

Since this system run offline with no one watching live, first thing is we could not depend on a person checking dashboard. We need automate checking/monitoring and alert if anything fails.

So keep test tiles inside the system, run this set again through model on a schedule, compare result(accuracy, confidence distribution, etc.) with our benchmark recorded when we deployed. If any big drop or drift, flag a row in db or update in dashboard, so whenever we open the system/dashboard they can know the fault and directly check this file/flag.

Also we can detect data distribution change automatically using statistical test, we can write script to test the data with users and data used for training and find out if any significant difference between them, then flag it.

Along with that, track flagged(review) prediction count, if it goes up, something wrong with model or data and proportion of each class in the result, may be a huge proportion increase of a certain class is red flag, we need to inspect.

Dashboard/log file is useful, but only helpful if someone actually check it. So build a self check that don't need person watching, dashboard is extra layer on top for whenever someone does look. Dashboard can helps to understand how many request served, how many failed, latency, etc.

3. Tiles are coming in fine, but the stored results look wrong. Walk us through how you'd find the cause — your actual steps, in order.

Answer:

Ok, so here the tiles have no issues, so we can avoid the chance for data distribution issues.

First step, re test the same tile again, test it outside the api, check if same wrong result come again or it was random. Also check is this happening for one sample or one class only, or happening for all result, this helps to understand where to look.

After that, inspect all component from preprocessing to post processing.

First check pre processing - make sure no unnecessary step affect model, and same transform object used in training is used here also. Then check model make sure correct/best trained weight loaded, also model set in eval mode so dropout/batchnorm not behave wrong. Then check post processing - logits and probability, selecting correct label properly.

Then check decision/validation layer, if threshold/margin comparison logic doing correct thing.

Then last, check db insertion code - make sure no default value getting saved instead of actual predicted value. May be wrong result is not a model problem at all, it is just wrong value getting written into db.

4. What's the weakest part of your design, and what would break it first?

Answer:

I built a simple PoC that work in local, not ready for production. So in real production there are many issues that come.

One issue is the assumption of single request. This is built based on the assumption one request at a time. If multiple request came, then the latency is uncontrollable or even more request can cause system crash due no proper backpressure. Also it is not a fully async system and sqlite does not support concurrent operations. So it fails in multiple user cases.

Another issue is, it does not have a proper monitoring or log files, so if anything crash hard to debug or even hard to detect an issue in this system.

Also the decision threshold like confidence threshold and margin threshold are assumed for now, not properly find out from real testing, so high chances to it fails.